



HUST

ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.



ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

FUNDAMENTALS OF OPTIMIZATION

Dynamic Programming

ONE LOVE. ONE FUTURE.

CONTENT

- Introduction to Dynamic Programming
- Examples

Introduction to Dynamic Programming

- R.E.Bellman (1920-1984)



RICHARD BELLMAN ON THE BIRTH OF DYNAMIC PROGRAMMING

STUART DREYFUS

University of California, Berkeley, IEOR, Berkeley, California 94720, dreyfus@ieor.berkeley.edu

What follows concerns events from the summer of 1949, when Richard Bellman first became interested in multistage decision problems, until 1955. Although Bellman died on March 19, 1984, the story will be told in his own words since he left behind an entertaining and informative autobiography, *Eye of the Hurricane* (World Scientific Publishing Company, Singapore, 1984), whose publisher has generously approved extensive excerpting.

During the summer of 1949 Bellman, a tenured associate professor of mathematics at Stanford University with a developing interest in analytic number theory, was consulting for the second summer at the RAND Corporation in Santa Monica. He had received his Ph.D. from Princeton in 1946 at the age of 25, despite various war-related activities during World War II—including being assigned by the Army to the Manhattan Project in Los Alamos. He had already exhibited outstanding ability both in pure mathematics and in solving applied problems arising from the physical world. Assured of a successful conventional academic career, Bellman, during the period under consideration, cast his lot instead with the kind of applied mathematics later to be known as operations research. In those days applied practitioners were regarded as distinctly second-class citizens of the mathematical fraternity. Always one to enjoy controversy, when invited to speak at various university mathematics department seminars, Bellman delighted in justifying his choice of applied over pure mathematics as being motivated by the real world's greater challenges and mathematical demands.

what RAND was interested in. He suggested that I work on multistage decision processes. I started following that suggestion" (p. 157).

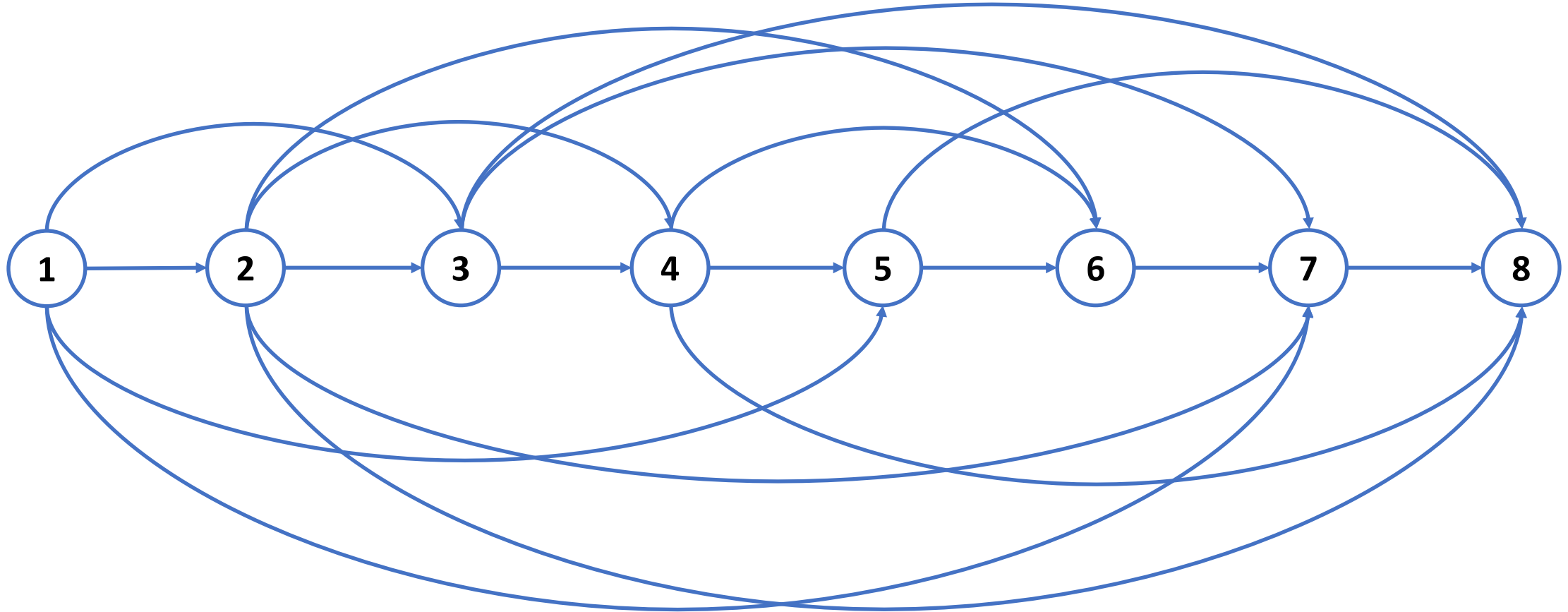
CHOICE OF THE NAME DYNAMIC PROGRAMMING

"I spent the Fall quarter (of 1950) at RAND. My first task was to find a name for multistage decision processes.

"An interesting question is, 'Where did the name, dynamic programming, come from?' The 1950s were not good years for mathematical research. We had a very interesting gentleman in Washington named Wilson. He was Secretary of Defense, and he actually had a pathological fear and hatred of the word, research. I'm not using the term lightly; I'm using it precisely. His face would suffuse, he would turn red, and he would get violent if people used the term, research, in his presence. You can imagine how he felt, then, about the term, mathematical. The RAND Corporation was employed by the Air Force, and the Air Force had Wilson as its boss, essentially. Hence, I felt I had to do something to shield Wilson and the Air Force from the fact that I was really doing mathematics inside the RAND Corporation. What title, what name, could I choose? In the first place I was interested in planning, in decision making, in thinking. But planning, is not a good word for various reasons. I decided therefore to use the word, 'programming.' I wanted to get across the idea that this was dynamic, this was multistage, this was time-varying—I thought, let's kill two birds with one stone. Let's take a word that has an

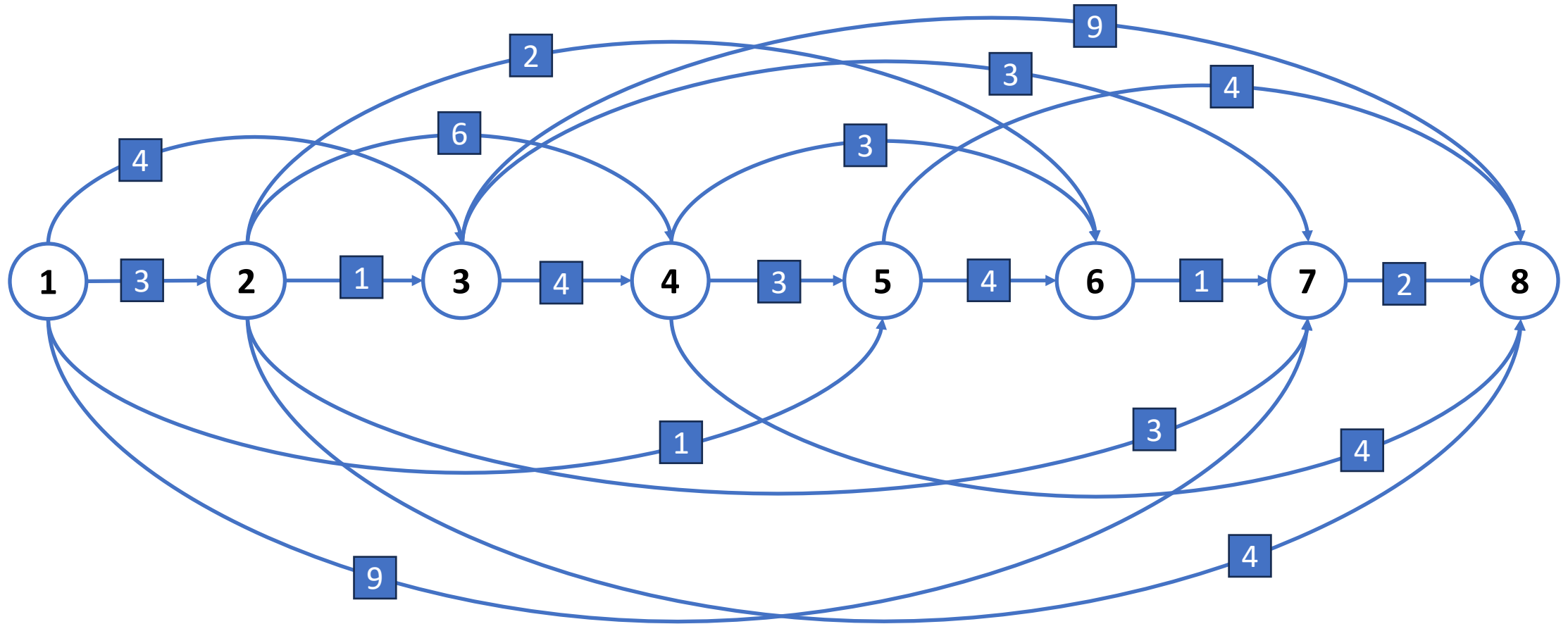
Introduction to Dynamic Programming

- How many ways to travel from point 1 to point 8?



Introduction to Dynamic Programming

- What is the shortest-length path to travel from point 1 to point 8?



Introduction to Dynamic Programming

- Dynamic programming
 - Divide the original problem into **overlapping** subproblems
 - Recurrence relations
 - Solution to a subproblem is specified based on solutions to smaller subproblems
 - Top-down approach (by recursion):
 - Solve subproblems are solved recursively
 - Base-case: solutions to the smallest subproblems are computed trivially (no recursion)
 - Bottom-up:
 - Smallest subproblems are solved trivially
 - Specify the solution to each subproblem based on the solutions to smaller subproblems (using recurrence relations)
 - ***Do not find the solution to the same problem more than one time (memorization)***

Examples – Money exchange

- Given a set of coins with denominations $D_1, D_2, \dots, D_n$ and an amount of money X . Find the minimum number of coins to exchange for X .
- Like the knapsack problem?
- Is there a greedy algorithm to solve this problem?
- The knapsack problem learned in Discrete Mathematics is solved using the branch and bound algorithm. The greedy algorithm is not certain of providing the optimal solution, and in many cases cannot even provide the solution...
- Try using the Dynamic Programming method!
- Finally, make comments on different approaches to solving this problem

Examples – Money exchange

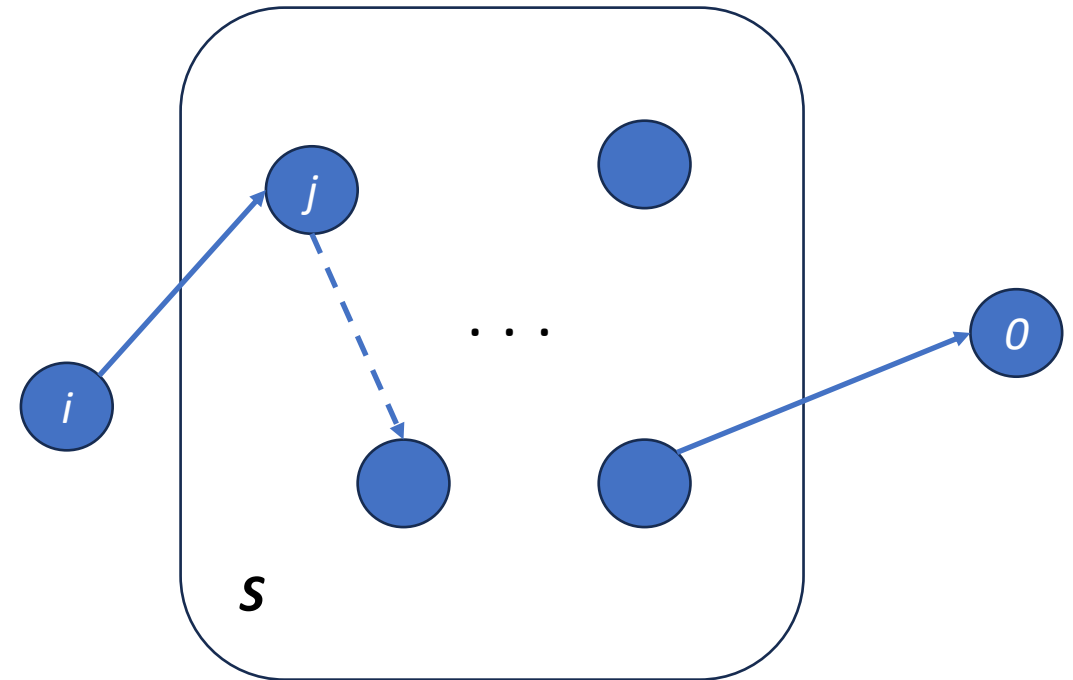
- Let $MinCoin(i, x)$ be the minimum amount of money needed to exchange denomination x if only the denominations $D_1, D_2, \dots, D_i$ are allowed to be used.
- Base case:
$$\begin{aligned}MinCoin(i, x) &= \infty \text{ nếu } x < 0 \\MinCoin(i, 0) &= 0 \\MinCoin(0, x) &= \infty\end{aligned}$$
- Recurrence relation:
$$MinCoin(i, x) = \min \begin{cases} 1 + MinCoin(i, x - D_i) \\ MinCoin(i - 1, x) \end{cases}$$

Examples – TSP

- Given a graph of n vertices $\{0, 1, \dots, n - 1\}$ and the weight value C_{ij} on each pair of vertices i, j . Find a cycle that goes through all the vertices of the graph, each vertex exactly once, so that the sum of the weights on that cycle is minimum.

Examples – TSP

- Without loss of generality, assume the cycle begins and ends at vertex 0.
- Let $TSP(i, S)$ be the shortest route starting from vertex i , visiting all vertices in S and returning to vertex 0.
- Base case: $TSP(i, \{\}) = C_{i,0}$
- Recurrence relation:
 - $TSP(i, S) = \min_{j \in S} \{ C_{i,j} + TSP(j, S \setminus \{j\}) \}$
- Solution to the original problem is $TSP(0, \{1, 2, \dots, n-1\})$



Examples – TSP

- Bitmasking:
 - A subset of $\{0, 1, 2, \dots, n-1\}$ is represented by a sequence of bits $b_{n-1}b_{n-2}\dots b_0$
 - $b_i = 1$ means that i belongs to the subset, and $b_i = 0$ means that i does not belong to the subset
 - The sequence $b_{n-1}b_{n-2}\dots b_0$ is represented by a non-negative integer N
 - Perform bit manipulation on N to reflect the corresponding set operations: insertion, removal, existence checking

```
unsigned int removeItem(unsigned int S, int i){
    return S ^ (1 << i);
}

unsigned int addItem(unsigned int S, int i){
    return S | (1 << i);
}
```

```
bool contains(unsigned int S, int i){
    if(((S >> i) & 1) > 0)
        return true;
    else return false;
}
```

Examples – TSP

```
int main(){
    scanf("%d",&n);
    for(int i = 0; i <= n-1; i++)
        for(int j = 0; j <= n-1; j++)
            scanf("%d",&d[i][j]);
    unsigned int S = 0;
    for(int i = 1; i <= n-1; i++)
        S = addItem(S,i);
    for(int i = 0; i < 32; i++)
        for(int j = 0; j < N; j++)
            M[i][j] = -1;
    cout << TSP(0,S);
}
```

```
int TSP(int i, unsigned int S){
    if(S == 0){ M[i][S] = d[i][0]; return M[i][S]; }
    if(M[i][S] == -1){// subproblem not solved
        M[i][S] = 100000000;
        for(int j = 0; j <= n-1; j++){
            if(contains(S,j)){
                int A = TSP(j,removeItem(S,j));
                if(M[i][S] > d[i][j] + A){ M[i][S] = d[i][j] + A; }
            }
        }
    }
    return M[i][S];
}
```

A decorative graphic on the left side of the slide. It features a dark blue background with a large, stylized circular pattern composed of many small red dots. The dots are arranged in concentric, slightly irregular rings, creating a sense of depth and movement. The word "HUST" is centered within this pattern.

HUST

THANK YOU !